Key Features and Elements of Reinforcement Learning (RL)

Reinforcement Learning (RL) is a goal-oriented learning framework where an agent interacts with an environment to learn an optimal policy by trial and error. The key elements of RL include **agents**, **environment**, **states**, **actions**, and **rewards**. Each plays a pivotal role in decision-making and achieving desired outcomes.

---

1. Agent

- **Definition**: The agent is the decision-maker or learner in the RL system. It selects actions based on its observations of the environment to maximize cumulative rewards over time.
- **Role**:
  - Learns and improves its policy through repeated interactions.
  - Uses exploration (to discover) and exploitation (to act optimally).

Industry Examples:

1. **Robotics**:
   - Agents in RL control robotic arms in assembly lines.
   - **Example**: Fanuc uses RL agents to optimize robotic assembly of electronic components, reducing errors and time.
2. **E-commerce**:
   - Agents recommend personalized products to users.
   - **Example**: Amazon's RL agents suggest items by learning user preferences over time.
3. **Gaming**:
   - Agents control virtual players to compete against humans or other agents.
   - **Example**: DeepMind's AlphaGo outperformed human champions in Go.

Practical Significance:

Agents act as the core of RL, enabling the automation of decision-making processes in dynamic and uncertain environments.

---

2. Environment

- **Definition**: The environment represents the external system with which the agent interacts. It provides observations and feedback based on the

agent's actions.

- **Role**:
  - Defines the rules of interaction.
  - Determines how states transition based on actions.

Industry Examples:

1. **Traffic Systems**:
   - The environment is a simulated or real-world traffic network.
   - **Example**: Lyft trains RL agents to manage traffic signals, minimizing congestion.
2. **Healthcare**:
   - The environment represents patient responses to treatments.
   - **Example**: RL-based environments help optimize treatment plans for chronic diseases like diabetes.
3. **Finance**:
   - The environment models stock market dynamics.
   - **Example**: JP Morgan Chase uses RL environments to predict stock price trends and make trading decisions.

Practical Significance:

The environment ensures that the agent learns realistic and transferable policies by modeling real-world constraints and dynamics.

---

3. States

- **Definition**: A state is a representation of the environment at a specific point in time. It includes all relevant information required for decision-making.
- **Role**:
  - Encapsulates the "current situation" the agent is in.
  - Drives the agent’s choice of action.

Industry Examples:

1. **Autonomous Vehicles**:
   - States include vehicle speed, position, and sensor readings.
   - **Example**: Tesla’s Autopilot system defines states using inputs like camera vision, radar data, and GPS.
2. **Customer Support**:
   - States represent the customer’s issue, history, and satisfaction level.
   - **Example**: Zendesk’s RL systems adjust support agent strategies based on live customer states.
3. **Manufacturing**:

- States represent machine status, operational conditions, and maintenance schedules.
- **Example**: GE monitors industrial turbines' states for predictive maintenance.

Practical Significance:

States provide the agent with the context needed to make informed decisions, ensuring relevance and precision in actions.

---

4. Actions

- **Definition**: An action is a decision or move made by the agent to influence the environment. Actions can be discrete (e.g., turn left or right) or continuous (e.g., adjust speed by 5 km/h).
- **Role**:
  - Directly impacts state transitions.
  - Enables the agent to explore and learn from the environment.

Industry Examples:

1. **Supply Chain Management**:
   - Actions include restocking levels, reorder timing, and logistics planning.
   - **Example**: Walmart uses RL to decide when and how much stock to reorder for maximum efficiency.
2. **Energy Systems**:
   - Actions involve turning on/off energy sources or adjusting power output.
   - **Example**: RL agents optimize wind turbine angles in renewable energy systems.
3. **Financial Services**:
   - Actions include buying, holding, or selling financial assets.
   - **Example**: RL-based trading bots at Robinhood make actions based on market conditions.

Practical Significance:

Actions determine how effectively an agent can adapt to the environment and achieve its objectives by testing various strategies.

---

5. Rewards

- **Definition**: Rewards are numerical signals received by the agent from the

environment as feedback for its actions. Positive rewards encourage desirable behaviors, while negative rewards discourage undesirable ones.

- **Role**:
  - Guides the agent toward optimal behavior.
  - Evaluates the immediate impact of an action.

Industry Examples:

1. **Retail**:
   - Rewards are based on increased sales or customer satisfaction.
   - **Example**: Shopify uses RL agents to maximize conversion rates on e-commerce websites.
2. **Smart Grids**:
   - Rewards are based on energy savings or cost reductions.
   - **Example**: RL systems in Germany's energy grid minimize electricity costs during peak usage.
3. **Marketing Campaigns**:
   - Rewards are measured by campaign engagement metrics (e.g., clicks or purchases).
   - **Example**: Coca-Cola uses RL to optimize ad placements for maximum ROI.

Practical Significance:

Rewards provide a measurable criterion for success, enabling the agent to refine its policy and achieve long-term goals.

Interactions Between the Key Elements

The RL workflow involves continuous interaction between these elements:

1. The **agent** observes the **state** of the **environment**.
2. Based on the state, the agent selects an **action**.
3. The environment responds to the action, transitioning to a new state and providing a **reward**.
4. The agent learns from this feedback, updating its policy to maximize cumulative rewards.

Industry Case Studies Combining All Elements

1. **Smart Warehousing**:

   - **Agent**: Robotic pickers.
   - **Environment**: Warehouse layout and inventory system.

- **States**: Robot positions, item locations, and congestion levels.
- **Actions**: Move to a shelf, pick an item, or place an item.
- **Rewards**: Minimized retrieval time and energy consumption.
- **Example**: Amazon's robotic fulfillment centers use RL to optimize item retrieval.

2. **Chatbot Optimization**:

   - **Agent**: Customer service chatbot.
   - **Environment**: Conversations with users.
   - **States**: User's query, sentiment, and engagement level.
   - **Actions**: Provide a response, ask a clarifying question, or escalate to a human.
   - **Rewards**: Increased customer satisfaction and reduced response time.
   - **Example**: Uber uses RL-based chatbots for automated customer support.

3. **Self-Driving Cars**:

   - **Agent**: Self-driving software.
   - **Environment**: Roads, traffic signals, and other vehicles.
   - **States**: Car speed, lane position, and distances to obstacles.
   - **Actions**: Accelerate, brake, or turn.
   - **Rewards**: Safe and timely completion of trips.
   - **Example**: Waymo uses RL to train its autonomous vehicle software.

What is Reinforcement Learning?

Before we dive into model-free and model-based RL, let's quickly understand **Reinforcement Learning (RL)**:

- Imagine you're training a dog. You want it to learn tricks like sitting or rolling over. You don't tell it exactly what to do, but whenever it does something right, you give it a treat (reward). Over time, the dog learns which actions lead to treats and starts repeating them.
- RL works the same way. An "agent" (like the dog) learns to interact with its "environment" (like your commands) by trying different actions and receiving feedback (rewards or penalties).

The Two Ways to Learn in RL

There are two main ways an agent can learn in RL:

1. **Model-Free RL** (Trial-and-Error Learning).
2. **Model-Based RL** (Learning and Using a Mental Map).

---

1. Model-Free RL: Learning by Trial and Error

In model-free RL, the agent learns purely by trying actions and seeing the outcome. It doesn't try to understand how the environment works—like figuring out rules or patterns. Instead, it focuses only on "what works."

Real-Life Analogy

- Think of a child playing a new video game for the first time without reading the instructions. The child randomly presses buttons, and over time, they notice which buttons lead to good results (like jumping or shooting) and which don't.
- The child doesn't know the game's rules—they just learn what works through trial and error.

Characteristics of Model-Free RL

- **No Understanding of Rules**: The agent doesn't build a map or model of how the environment works.
- **Direct Learning**: It learns directly from rewards and penalties.
- **Trial and Error**: The agent learns by trying different actions repeatedly.

Examples

1. **Gaming**:
    - AI agents like those in Atari games (e.g., Pac-Man or Breakout) learn purely by playing the game and maximizing their scores.
    - They don't understand the game's rules but learn which moves earn the most points.
2. **Robotics**:
    - A robot learning to walk doesn't start with knowledge of physics. It tries different movements, learns which ones help it stand and move, and gradually improves.

---

2. Model-Based RL: Learning and Using a Mental Map

In model-based RL, the agent creates a "mental map" of how the environment works. It learns the rules or patterns and uses this knowledge to make decisions.

Real-Life Analogy

- Imagine you're trying to navigate a new city. Instead of randomly walking around, you look at a map. The map shows you which roads lead to your destination. You use this map to plan your route before you start moving.
- In model-based RL, the agent builds its own "map" of the environment by exploring it. Then, it uses this map to plan the best actions.

Characteristics of Model-Based RL

- **Understanding Rules**: The agent tries to understand how the environment behaves (e.g., what happens when it takes an action).
- **Planning**: Before acting, the agent simulates possible actions to choose the best one.
- **Efficient Learning**: Because it plans ahead, it can learn faster than model-free RL.

Examples

1. **Healthcare**:
   - A system predicting the effect of different treatments on a patient's health. Instead of trying random treatments, it simulates outcomes using a medical model and selects the best option.
2. **Autonomous Cars**:
   - Self-driving cars predict the behavior of other vehicles and plan their movements (e.g., overtaking safely) instead of randomly driving and learning from mistakes.

Key Differences Between Model-Free and Model-Based RL

Feature	Model-Free RL	Model-Based RL
How it Learns	Learns directly from actions and rewards (trial-and-error).	Builds a mental map of the environment to plan actions.
Rules or Patterns	Doesn't try to understand the environment's rules.	Learns how the environment behaves.
Speed	Slow because it learns everything from scratch.	Faster because it plans ahead.
Examples	AI in video games, robots learning to walk.	Self-driving cars, healthcare systems.

In-Depth Example: Navigating a Maze

Model-Free RL

- A robot starts in a maze. It has no idea of the maze's layout or where the

exit is.
- It randomly moves, hitting walls, until it finds a path that leads to the exit.
- Over time, it remembers which moves worked and learns to reach the exit faster.

Model-Based RL

- Another robot enters the same maze. Instead of randomly moving, it explores a bit and creates a map of the maze.
- Using this map, it plans the shortest path to the exit.
- It reaches the exit much faster because it used planning.

When to Use Model-Free vs. Model-Based RL

1. **Use Model-Free RL When**:
    - The environment is too complex to model (e.g., learning to play a video game).
    - It's okay to spend a lot of time learning through trial and error.
2. **Use Model-Based RL When**:
    - Building a model is feasible (e.g., creating a map or a simulation).
    - Interacting with the real environment is expensive or risky (e.g., healthcare, autonomous driving).

Combined Approach

Some methods, like **Dyna-Q**, combine the strengths of both:

- The agent builds a partial map of the environment but still learns through trial and error.
- This hybrid approach is useful in real-world systems where modeling is possible but incomplete.

Conclusion

The choice between model-free and model-based RL depends on the problem you're solving.

- **Model-Free RL** is like a novice learning by doing.
- **Model-Based RL** is like an expert planning ahead.

By understanding these approaches, you can decide which method suits your needs—whether it's training an AI for games, optimizing supply chains, or

managing healthcare systems.

Model-free reinforcement learning (RL)

Model-free reinforcement learning (RL) consists of methods where the agent learns directly from its interactions with the environment, without explicitly building or using a model of the environment's dynamics (state transitions or reward function). The agent focuses on learning a policy or value function purely through trial-and-error.

Model-Free Methods

Model-free RL can be broadly divided into two categories:

1. Value-Based Methods

- These methods focus on learning value functions, which estimate how good it is to be in a certain state or to take a certain action in a state.

2. Policy-Based Methods

- These methods directly learn the policy (a mapping from states to actions) without explicitly estimating value functions.

3. Actor-Critic Methods (Hybrid)

- Combines value-based and policy-based methods by learning both the policy (actor) and the value function (critic).

1. Value-Based Methods

Value-based methods aim to estimate the value of:

- **State-value function ($V(s)$)**: The expected total reward from starting in state s.
- **Action-value function ($Q(s,a)$)**: The expected total reward from starting in state s, taking action a, and following the optimal policy.

Examples of Value-Based Methods

1. **Q-Learning**:
   - Learns the $Q(s,a)$ function iteratively using the formula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

   - **Key Idea**: Finds the optimal action-value function (Q^*) for choosing the best actions.
   - **Application**: Games, robotics, and navigation tasks.

2. **SARSA (State-Action-Reward-State-Action)**:
   - Similar to Q-learning but updates $Q(s, a)$ based on the action the agent is currently taking:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma Q(s', a') - Q(s, a) \right]$$

   - **Key Difference from Q-Learning**: SARSA is an **on-policy** algorithm (follows its current policy while learning), whereas Q-learning is **off-policy**.
3. **Deep Q-Learning (DQN)**:
   - Extends Q-Learning by using deep neural networks to approximate the $Q(s, a)$ function.
   - **Significance**: Handles large or continuous state spaces.
   - **Applications**: Complex games like Atari, robotics, and autonomous driving.

2. Policy-Based Methods

Policy-based methods aim to learn the policy ($\pi(a|s)$) directly without estimating value functions.

Examples of Policy-Based Methods

1. **REINFORCE Algorithm**:
   - A basic policy gradient method that uses the gradient of the expected reward to improve the policy.
   - **Update Rule**:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a|s) R$$

     Where:
     - θ: Parameters of the policy.
     - R: Total reward obtained from the episode.

2. **Proximal Policy Optimization (PPO)**:
   - A more advanced policy gradient method that stabilizes learning by limiting how much the policy changes in each update.
   - **Applications**: Robotics, games, and continuous control tasks.
3. **Soft Actor-Critic (SAC)**:
   - A policy gradient method that incorporates an entropy term to encourage exploration.

3. Actor-Critic Methods

These methods combine the strengths of value-based and policy-based approaches:

- **Actor**: Learns the policy ($\pi(a|s)$) to decide actions.
- **Critic**: Learns the value function ($V(s)$ or $Q(s,a)$) to evaluate the actions taken by the actor.

Examples of Actor-Critic Methods

1. **Advantage Actor-Critic (A2C)**:
   - Uses the "advantage" function:
     $$A(s,a) = Q(s,a) - V(s)$$
     To stabilize policy updates.
2. **Deep Deterministic Policy Gradient (DDPG)**:
   - Extends actor-critic methods to handle continuous action spaces.
3. **Twin Delayed Deep Deterministic Policy Gradient (TD3)**:
   - Improves DDPG by reducing overestimation bias in Q-values.

Comparison of Key Model-Free Methods

Method	Type	On-Policy/Off-Policy	Use Case
Q-Learning	Value-Based	Off-Policy	Discrete action spaces, navigation tasks.
SARSA	Value-Based	On-Policy	Safer exploration (e.g., robot learning).
DQN	Value-Based	Off-Policy	Games, robotics (large state spaces).
REINFORCE	Policy-Based	On-Policy	Continuous control, stochastic policies.
PPO	Policy-Based	On-Policy	Robotics, games (robust performance).
A2C	Actor-Critic	On-Policy	Stabilized policy updates.
DDPG	Actor-Critic	Off-Policy	Continuous action spaces (e.g., driving).

How to Teach Model-Free RL

1. **Start with Simple Concepts**:
   - Begin with **Q-Learning** because it's intuitive and uses a table to store values.
   - Demonstrate learning through trial-and-error in a simple environment like **GridWorld**.
2. **Progress to Neural Networks**:
   - Introduce **DQN** to show how deep learning can approximate Q-values in complex environments.
   - Use visualizations (e.g., game AI learning over time) to engage participants.
3. **Introduce Policy-Based Methods**:
   - Explain situations where value-based methods fail, such as in continuous action spaces.
   - Use algorithms like **REINFORCE** and **PPO** to show how policies can be optimized directly.
4. **Highlight Real-World Applications**:
   - Value-Based: Robot navigation, playing board games.
   - Policy-Based: Continuous control tasks like flying drones.
   - Actor-Critic: Complex simulations like resource allocation.

What is Q-Learning?

Q-Learning is a **model-free reinforcement learning algorithm**. This means:

- The agent **does not need to know the rules or dynamics** of the environment beforehand.
- It learns **which actions to take in different situations (states)** by trial and error, aiming to maximize its total rewards over time.

Imagine you're playing a video game for the first time:

1. You don't know the rules or the best strategies.
2. You try different moves, observe what works, and get better with practice.
3. Over time, you learn which actions (e.g., jumping, attacking) are best in specific situations.

Q-Learning works the same way! It allows an agent to learn the "value" of taking an action in a particular state.

Key Concepts in Q-Learning

1. Agent, Environment, States, and Actions

- **Agent**: The learner or decision-maker (e.g., a robot, game character).
- **Environment**: The world the agent interacts with (e.g., a grid, a game).
- **State (s)**: A situation the agent is in (e.g., the robot's current position in the grid).
- **Action (a)**: A move the agent can make (e.g., go left, right, up, or down).

2. Reward (r)

- A **numerical value** given to the agent as feedback.
- The goal of the agent is to maximize its cumulative rewards over time.
  - Example: In a maze, reaching the goal might give a reward of $+10$, and hitting a wall might give 0.

3. Q-Value ($Q(s, a)$)

- The **Q-value** tells the agent how good it is to take a specific action a in a state s.
- **Goal**: The agent learns these Q-values through interaction.

How Does Q-Learning Work?

Q-Learning uses a simple **mathematical formula** to update the Q-values for state-action pairs. Let's break it down step by step:

The Q-Value Update Formula

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

What Does This Mean?

1. **Current Q-Value ($Q(s, a)$)**:
   - The agent's current guess of how good it is to take action a in state s.
2. **Reward (r)**:
   - The immediate reward received after taking action a.
3. **Future Q-Value ($\max_{a'} Q(s', a')$)**:
   - The estimated value of the best action in the next state s'.
4. **Learning Rate (α)**:
   - A number between 0 and 1 that controls how much the new information overrides the old value.
5. **Discount Factor (γ)**:
   - A number between 0 and 1 that controls how much importance is given to future rewards compared to immediate rewards.

Q-Learning Algorithm in Steps

1. **Initialize** the Q-table:
   - Create a table with all possible states and actions, and set all Q-values to 0 (or small random values).
2. **Repeat for each episode** (learning trial):
   - Start in an initial state.
   - For each step:
     - **Choose an action**:
       - Use an **exploration-exploitation strategy** like ϵ-greedy (explained later).
     - **Take the action** and observe the next state and reward.
     - **Update the Q-value** using the formula above.
     - Move to the next state.
3. **Stop** when the agent has learned the optimal policy (best actions in all states).

Key Concept: Exploration vs. Exploitation

The agent faces a dilemma:

- **Explore**: Try new actions to discover potentially better rewards.
- **Exploit**: Use the best-known action to maximize rewards. This is handled using an **ϵ-greedy strategy**:
- With probability ϵ, choose a random action (exploration).
- With probability $1 - \epsilon$, choose the action with the highest Q-value (exploitation).

Example: Q-Learning in a GridWorld

Problem Setup

Imagine a 4x4 grid:

- Start: Top-left $(0, 0)$.
- Goal: Bottom-right $(3, 3)$.
- Obstacles: Certain cells have 0 reward or penalties.

+-----+-----+-----+ | (0,0) | (0,1) | (0,2) | +-----+-----+-----+ | (1,0) | (1,1) | (1,2) | +-----+-----+-----+ | (2,0) | (2,1) | (2,2) | +-----+-----+-----+

In [3]:
```
import numpy as np  # Importing the NumPy library for numerical com
import random  # Importing the random module for generating random

# Step 1: Create the GridWorld environment
class GridWorld:
    def __init__(self, size=4):
        # Initialize the grid size (default is 4x4)
        self.size = size
        # Define the starting position of the agent (top-left corne
        self.start = (0, 0)
```

```python
        # Define the goal position (bottom-right corner)
        self.goal = (3, 3)
        # Set the current state to the starting position
        self.state = self.start

    def reset(self):
        # Reset the agent's position to the starting point
        self.state = self.start
        return self.state  # Return the starting state

    def step(self, action):
        # Get the current row and column of the agent
        row, col = self.state
        # Move the agent up, if possible
        if action == "up" and row > 0:
            row -= 1
        # Move the agent down, if possible
        elif action == "down" and row < self.size - 1:
            row += 1
        # Move the agent left, if possible
        elif action == "left" and col > 0:
            col -= 1
        # Move the agent right, if possible
        elif action == "right" and col < self.size - 1:
            col += 1
        # Update the next state after the move
        next_state = (row, col)

        # Check if the agent has reached the goal
        if next_state == self.goal:
            reward = 10  # Give a reward of 10 for reaching the goa
            done = True  # Mark the episode as finished
        else:
            reward = -1  # Give a penalty of -1 for each step taken
            done = False  # The episode is not yet finished

        # Update the current state to the new state
        self.state = next_state
        # Return the next state, the reward, and whether the episod
        return next_state, reward, done

# Step 2: Q-Learning Algorithm
def q_learning(env, episodes, alpha=0.1, gamma=0.9, epsilon=0.1):
    # Define the possible actions the agent can take
    actions = ["up", "down", "left", "right"]
    # Initialize the Q-table as a dictionary
    Q = {}
    for row in range(env.size):  # Loop over each row in the grid
        for col in range(env.size):  # Loop over each column in the
            # Initialize Q-values for all actions in each state to
            Q[(row, col)] = {action: 0 for action in actions}
#             print("Q[(row, col)]:",Q[(row, col)])

    # Loop through the specified number of episodes
    for episode in range(episodes):
        # Reset the environment to the starting state
```

```
        state = env.reset()
        done = False  # Flag to indicate if the episode is finished

        while not done:
            # Decide whether to explore or exploit using epsilon-gr
            if random.uniform(0, 1) < epsilon:
                # Exploration: Randomly choose an action
                action = random.choice(actions)
            else:
                # Exploitation: Choose the action with the highest 
                action = max(Q[state], key=Q[state].get)

            # Take the chosen action and observe the next state, re
            next_state, reward, done = env.step(action)

            # Find the best action's Q-value for the next state
            best_next_action = max(Q[next_state], key=Q[next_state]
            # Update the Q-value for the current state-action pair
            Q[state][action] += alpha * (reward + gamma * Q[next_st

            # Move to the next state
            state = next_state

    # Return the learned Q-table
    return Q

# Step 3: Run the Algorithm
# Create an instance of the GridWorld environment
env = GridWorld()
# Train the agent using Q-Learning for 500 episodes
Q = q_learning(env, episodes=500)

# Step 4: Display Results
# Print the learned Q-values for all states and actions
print("Learned Q-Values:")
for state, actions in Q.items():
    # Print the Q-values for each state
    print(f"State {state}: {actions}")
```

```
Learned Q-Values:
State (0, 0): {'up': -0.1290900012222771, 'down': -0.343063586651974
34, 'left': -0.6748403648245014, 'right': 1.8097999999999863}
State (0, 1): {'up': 0.6442578641583471, 'down': 3.1219999999999857,
'left': -0.01901506727130879, 'right': -0.18735236669730013}
State (0, 2): {'up': -0.5072249680517285, 'down': 2.474892593840107
4, 'left': -0.4193801263661908, 'right': -0.6256432000000001}
State (0, 3): {'up': -0.4813668100000003, 'down': -0.43462900000000
004, 'left': -0.5223076261508005, 'right': -0.490099501}
State (1, 0): {'up': -1.176943202082976, 'down': -1.121750931192948
3, 'left': -1.22026649057471, 'right': 2.509011117470379}
State (1, 1): {'up': 1.017560942947478, 'down': 4.579999999999986, '
left': -0.2379306694668606, 'right': 2.661110694450599}
State (1, 2): {'up': -0.3043, 'down': 6.054909716831699, 'left': 0.0
20766668638601993, 'right': -0.2881}
State (1, 3): {'up': -0.28891, 'down': 3.5354649662799247, 'left': -
0.21520000000000003, 'right': -0.199}
State (2, 0): {'up': -0.659817700108435, 'down': -0.62077348, 'lef
t': -0.7558545180537874, 'right': 3.0209958435707693}
State (2, 1): {'up': 2.4656776968464764, 'down': 0.4418506795124053,
'left': 0.33129821268821635, 'right': 6.199999999999989}
State (2, 2): {'up': 2.168610057118634, 'down': 4.162869352024352, '
left': 3.1414046850640673, 'right': 7.999999999999991}
State (2, 3): {'up': 0.24897579643078915, 'down': 9.999999999999993,
'left': 5.075750472282999, 'right': 6.169208300554489}
State (3, 0): {'up': -0.416163592, 'down': -0.3940399, 'left': -0.39
40399, 'right': -0.4528540000000003}
State (3, 1): {'up': 0.2781709999995656, 'down': -0.199, 'left': -0.
21601000000000004, 'right': 3.745947273904453}
State (3, 2): {'up': -0.1, 'down': 0.5749148282327008, 'left': -0.10
900000000000001, 'right': 9.282102012308147}
State (3, 3): {'up': 0, 'down': 0, 'left': 0, 'right': 0}
```

Why Is the Number of Episodes Important?

- **Episodes** represent individual trials where the agent starts at the initial state and attempts to reach the goal while learning from its actions.
- The number of episodes determines how much experience the agent gains:
  - **Fewer episodes**: The agent may not explore enough of the environment to learn an optimal policy.
  - **More episodes**: The agent has more opportunities to explore, learn from mistakes, and converge to the best possible strategy.

Why Choose 500 Episodes?

1. **Balancing Exploration and Convergence**:

    - 500 episodes is often a good compromise between allowing the

agent to explore the environment and giving it enough time to converge to an optimal policy.
    - Too few episodes may result in incomplete learning, while too many may waste computation time if the policy has already converged.
2. **Grid Size and Complexity**:
    - For a **4x4 grid**, the environment is relatively simple, with only 16 states and 4 possible actions per state.
    - 500 episodes provide sufficient iterations for the agent to explore most (if not all) states and actions multiple times.
3. **Learning Rate ((\alpha)) and Discount Factor ((\gamma))**:
    - With the given parameters:
        - **(\alpha = 0.1)** (small learning rate): Learning is gradual, requiring more episodes to improve Q-values.
        - **(\gamma = 0.9)** (discount factor): Rewards from future states are heavily considered, requiring more episodes to propagate these values backward.
4. **Trial-and-Error Nature of Q-Learning**:
    - Since Q-Learning is based on trial and error, the agent needs multiple episodes to discover the best actions in different states, especially when using **epsilon-greedy exploration** (random actions occur in some episodes).

How to Choose the Number of Episodes

The number of episodes should depend on:

1. **Environment Complexity**:
    - Larger environments or more complex tasks may require significantly more episodes.
    - Example: A **10x10 grid** will need more episodes than a **4x4 grid** due to the increased state-action space.
2. **Convergence Check**:
    - If the Q-values or rewards stabilize (stop changing significantly) before reaching 500 episodes, fewer episodes could suffice.
    - Conversely, if the agent hasn't learned an optimal policy by the 500th episode, more episodes might be needed.
3. **Computational Efficiency**:
    - In practical scenarios, the number of episodes is often limited by available computation time or resources.

How to Test If 500 Is Optimal

1. **Experiment with Fewer Episodes**:
    - Run the algorithm with 100, 200, or 300 episodes and check if the learned policy (Q-values) is significantly different.
    - If the policy converges to the same results with fewer episodes, 500 may be excessive.
2. **Monitor Convergence**:
    - Track the total rewards or Q-values during training. If these stabilize early, you can stop training.

Applications of Q-Learning

1. **Game AI**:
    - Teaching AI agents to play games like Snake, Tetris, or Pac-Man.
2. **Robotics**:
    - Robots learning to navigate obstacles in warehouses.
3. **Traffic Systems**:
    - Optimizing traffic lights to reduce congestion.
4. **Finance**:
    - Learning optimal trading strategies.

In [ ]:

```
```

Q-Learning for the Wumpus World Problem

Problem Statement

The **Wumpus World** is a grid-based world where an agent explores and tries to find gold while avoiding dangers such as pits and the fearsome Wumpus. The agent must decide its actions based on limited knowledge and rewards for achieving specific goals.

Objective

- Navigate the grid to find the **gold**.
- Avoid falling into **pits** or being killed by the **Wumpus**.
- Use **Q-Learning** to train the agent to explore the grid and maximize its reward.

Components

1. **States**:
   - The agent's position in the grid.
   - Whether the agent perceives indicators like breeze (near pits) or stench (near Wumpus).
2. **Actions**:
   - Move **up**, **down**, **left**, or **right**.
3. **Rewards**:
   - **+10** for finding gold.
   - **-100** for falling into a pit or encountering the Wumpus.
   - **-1** for each step taken to encourage shorter solutions.

In [6]:
```
# The output of this code changes everytime you execute the code
import numpy as np
import random

# Step 1: Define the Wumpus World Environment
class WumpusWorld:
    def __init__(self, grid_size=4):
        """
        Initialize the Wumpus World environment.
        """
        self.grid_size = grid_size  # Define the size of the grid (
        self.reset()

    def reset(self):
        """
        Reset the environment to the initial state.
        """
        self.agent_position = (0, 0)  # Start at the top-left corne
        self.gold_position = (3, 3)  # Gold is at the bottom-right
        self.pit_positions = [(1, 2), (2, 1)]  # Define pit locatio
        self.wumpus_position = (2, 2)  # Define the Wumpus position
        return self.agent_position  # Return the initial position o

    def step(self, action):
        """
        Take an action and update the environment.
        """
        x, y = self.agent_position
        if action == "up" and x > 0:
            x -= 1
        elif action == "down" and x < self.grid_size - 1:
            x += 1
        elif action == "left" and y > 0:
            y -= 1
        elif action == "right" and y < self.grid_size - 1:
            y += 1

        self.agent_position = (x, y)
```

```
        # Calculate reward
        if self.agent_position == self.gold_position:
            return self.agent_position, 10, True  # Found gold
        elif self.agent_position in self.pit_positions or self.agen
            return self.agent_position, -100, True  # Fell into a p
        else:
            return self.agent_position, -1, False  # Regular step

# Step 2: Q-Learning Algorithm
def q_learning(env, episodes, alpha=0.1, gamma=0.9, epsilon=0.1):
    """
    Perform Q-learning to train the agent.
    """
    actions = ["up", "down", "left", "right"]  # Define possible ac
    Q = {}  # Initialize Q-table

    # Initialize Q-values for all state-action pairs
    for x in range(env.grid_size):
        for y in range(env.grid_size):
            Q[(x, y)] = {action: 0 for action in actions}

    # Training loop
    for episode in range(episodes):
        state = env.reset()  # Reset the environment at the start o
        done = False

        while not done:
            # Epsilon-greedy action selection
            if random.uniform(0, 1) < epsilon:
                action = random.choice(actions)  # Explore
            else:
                action = max(Q[state], key=Q[state].get)  # Exploit

            # Take the action and observe the result
            next_state, reward, done = env.step(action)

            # Update Q-value using the Q-learning formula
            best_next_action = max(Q[next_state], key=Q[next_state]
            Q[state][action] += alpha * (reward + gamma * Q[next_st

            # Move to the next state
            state = next_state

    return Q

# Step 3: Run the Algorithm
env = WumpusWorld(grid_size=4)
episodes = 500  # Define the number of training episodes
Q = q_learning(env, episodes, alpha=0.1, gamma=0.9, epsilon=0.1)

# Step 4: Display Results
print("Learned Q-Values for the Wumpus World:")
for state, actions in list(Q.items())[:10]:  # Display a sample of
    print(f"State {state}: {actions}")
```

```
Learned Q-Values for the Wumpus World:
State (0, 0): {'up': -0.5180622750700713, 'down': -2.076366028180368
4, 'left': -0.31740030962492727, 'right': 1.8097999999999543}
State (0, 1): {'up': 1.1126077271792019, 'down': -1.774659634839603
8, 'left': -0.014190038828718957, 'right': 3.1219999999999835}
State (0, 2): {'up': 1.988929316136138, 'down': -86.49148282327009,
'left': 0.6830502730202942, 'right': 4.579999999999986}
State (0, 3): {'up': 2.7998241210651456, 'down': 6.199999999999989,
'left': 1.6033091628021396, 'right': 2.3212146549190518}
State (1, 0): {'up': -0.23829947877066815, 'down': -1.98402117434703
17, 'left': -2.1399385193625107, 'right': -2.057845199119823}
State (1, 1): {'up': 0.238844071654323, 'down': -10.0, 'left': -2.08
76763410891006, 'right': -34.39}
State (1, 2): {'up': 0, 'down': 0, 'left': 0, 'right': 0}
State (1, 3): {'up': 3.2170415339054435, 'down': 7.999999999999991,
'left': -74.58134171671, 'right': 4.91643647101299}
State (2, 0): {'up': -1.4587083615568703, 'down': -0.737366763564680
3, 'left': -1.4385303814633057, 'right': -19.0}
State (2, 1): {'up': 0, 'down': 0, 'left': 0, 'right': 0}
```

Interpreting the Q-Values for the Wumpus World

The output contains the learned **Q-values** for different states in the Wumpus World. These Q-values represent the agent's expected cumulative reward for taking each possible action (`'up'` , `'down'` , `'left'` , `'right'`) in a specific state.

Each state (e.g., `(0, 0)`) is associated with a set of Q-values for the available actions. By comparing these values, we can determine the best action to take in each state to maximize rewards.

Key Insights from the Q-Values

State (0, 0):

```
{'up': -0.518, 'down': -2.076, 'left': -0.317, 'right':
1.810}
```

- **Interpretation**:
  - **'right'** has the highest Q-value (`1.810`), indicating that moving **right** is the most beneficial action from this state.
  - Actions like **'up'**, **'down'**, and **'left'** have negative Q-values, suggesting penalties or less favorable outcomes.
- **What it Means**:
  - The agent has learned that moving **right** from `(0, 0)` leads closer to the goal and avoids danger.

State (0, 1):

```
{'up': 1.113, 'down': -1.775, 'left': -0.014, 'right':
3.122}
```

- **Interpretation**:
  - **'right'** has the highest Q-value (`3.122`), making it the optimal action.
  - **'up'** has a moderately positive Q-value (`1.113`), meaning it is less favorable than **'right'** but still a viable choice.
  - **'down'** and **'left'** have negative or low Q-values, indicating penalties.
- **What it Means**:
  - Moving **right** is the best action, while **'up'** might be a backup if the agent is forced to explore.

State (0, 2):

```
{'up': 1.989, 'down': -86.491, 'left': 0.683, 'right':
4.580}
```

- **Interpretation**:
  - **'right'** has the highest Q-value (`4.580`), leading closer to the goal.
  - **'down'** has a very large negative Q-value (`-86.491`), indicating a significant penalty (e.g., a pit or Wumpus).
  - **'up'** and **'left'** are moderately positive but less favorable than **'right'**.
- **What it Means**:
  - The agent avoids moving **down** due to the danger and prefers **right** to progress toward the goal.

State (0, 3):

```
{'up': 2.800, 'down': 6.200, 'left': 1.603, 'right': 2.321}
```

- **Interpretation**:
  - **'down'** has the highest Q-value (`6.200`), indicating that it likely leads directly to the goal.
  - **'up'**, **'left'**, and **'right'** have lower but positive Q-values.
- **What it Means**:
  - The agent strongly favors moving **down**, which likely results in reaching the goal.

State (1, 0):

```
{'up': -0.238, 'down': -1.984, 'left': -2.140, 'right':
-2.058}
```

- **Interpretation**:
  - All actions have negative Q-values, meaning the agent experiences penalties regardless of its choice.
  - **'up'** has the least negative Q-value (`-0.238`), suggesting it is the least harmful option.
- **What it Means**:
  - The agent avoids this state if possible but chooses **'up'** if forced to act.

State (1, 3):

```
{'up': 3.217, 'down': 8.000, 'left': -74.581, 'right':
4.916}
```

- **Interpretation**:
  - **'down'** has the highest Q-value (`8.000`), indicating it leads directly to the goal.
  - **'left'** has a very large negative Q-value (`-74.581`), indicating a high penalty (likely due to danger).
  - **'right'** and **'up'** are positive but less favorable than **'down'**.
- **What it Means**:
  - The agent avoids **'left'** and prefers **'down'** to reach the goal.

Zero Q-Values (States like (1, 2) and (2, 1))

- **Interpretation**:
  - These states have Q-values of `0` , meaning the agent either:
    - Did not explore these states during training.
    - Avoided these states due to penalties in nearby transitions.

General Observations

1. **Risk-Averse Behavior**:
   - The agent learns to avoid states or actions with high penalties (e.g., `(0, 2)` with `'down': -86.491`).
2. **Optimal Policy**:
   - For each state, the action with the highest Q-value represents the

optimal choice, guiding the agent toward the goal while minimizing risks.

3. **Exploration vs. Exploitation**:
    - States with incomplete exploration (e.g., zero Q-values) suggest the need for better balance in the agent's exploration and exploitation.

What Have We Achieved?

- **Dynamic Learning**:
    - The agent learns an optimal policy for navigating the grid by balancing exploration and exploitation.
- **Risk Management**:
    - The agent avoids high-risk areas like pits and the Wumpus.
- **Goal-Oriented Behavior**:
    - The agent consistently finds the shortest and safest path to the goal.

Practical Significance

1. **Path Optimization**:
    - This approach is applicable in robotics, where agents navigate unknown or hazardous environments.
2. **Risk-Aware AI**:
    - Similar techniques can be applied in AI systems requiring risk assessment, such as autonomous vehicles.
3. **Game Design**:
    - AI-driven NPCs in games can learn optimal strategies in maze-like environments.

SARSA (State-Action-Reward-State-Action) in Reinforcement Learning

SARSA is a popular on-policy reinforcement learning algorithm used to find the optimal policy for decision-making in an environment.

What is SARSA?

SARSA stands for:

1. **State**: The current position or condition of the agent in the environment.
2. **Action**: The decision or move the agent takes based on the current state.
3. **Reward**: The feedback (positive or negative) the agent receives after taking an action.
4. **Next State**: The resulting state after the agent takes the action.
5. **Next Action**: The next decision the agent makes from the resulting state.

SARSA is called an **on-policy algorithm** because it updates the value of the current state-action pair based on the agent's current policy (how it chooses actions).

How SARSA Works

SARSA updates its **Q-values** (state-action values) using the following equation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot (r + \gamma \cdot Q(s',a') - Q(s,a))$$

Where:

- $Q(s,a)$: Current Q-value for state s and action a.
- α: Learning rate (how much the agent adjusts its knowledge).
- r: Reward received after taking action a in state s.
- γ: Discount factor (how much the agent values future rewards).
- $Q(s',a')$: Q-value of the next state s' and the next action a'.

Key Features of SARSA

1. **On-Policy**: SARSA uses the agent’s current policy to choose actions while updating Q-values.
    - Example: If the agent decides to explore randomly 10% of the time, the update considers this behavior.
2. **Safe Learning**: Because it considers the agent's policy, SARSA is more cautious than off-policy algorithms like Q-learning.
3. **Balancing Exploration and Exploitation**: SARSA works well in environments where the agent must balance trying new things (exploration) and sticking to the best-known actions (exploitation).

SARSA in Simple Terms

Imagine you're a robot navigating a grid with rewards and penalties:

1. You start in a particular square (**state**).
2. You decide to move in a direction (**action**).
3. You land in a new square (**next state**) and receive a reward or penalty (**reward**).
4. Based on your current policy, you decide your next move (**next action**).
5. You update your Q-value for the current state and action based on what you learned.

SARSA vs. Q-Learning

Aspect	SARSA	Q-Learning
Type	On-policy	Off-policy
Policy Used	Updates Q-values using the agent's current policy.	Updates Q-values assuming the best policy.
Exploration	Considers exploration in updates.	Updates assume exploitation (greedy actions).
Safety	More cautious; avoids risky actions.	May take risky actions for maximum reward.

Real-World Example of SARSA

Scenario: Autonomous Vehicle at an Intersection

- **States**: The vehicle's position (e.g., intersection, straight road, or turn).
- **Actions**: Turn left, turn right, or go straight.
- **Reward**: Positive reward for reaching the destination quickly and safely; negative reward for collisions or delays.
- **Next State and Action**: Based on the current policy (e.g., if the policy says "explore," it may randomly try an unfamiliar route).

SARSA Implementation

Problem Statement

Imagine a **4x4 grid world** where an agent (a robot) needs to reach a specific **goal** at the bottom-right corner (state `(3, 3)`) from the top-left corner `(0, 0)`. The environment penalizes the agent for every step it takes (`-1`), encouraging it to find the shortest path. Once the agent reaches the goal, it gets a positive reward (`+10`), and the episode ends.

The agent learns how to navigate the grid using the **SARSA (State-Action-Reward-State-Action)** algorithm. The objective is to train the agent to discover the **optimal policy** (best set of actions) for reaching the goal efficiently, avoiding unnecessary steps.

The Environment

- **States**: Each cell in the grid represents a state. For a 4x4 grid, there are 16 states `(0, 0)` to `(3, 3)`.
- **Actions**: The agent can move:
  - **Up** (if it's not at the top edge).
  - **Down** (if it's not at the bottom edge).
  - **Left** (if it's not at the left edge).
  - **Right** (if it's not at the right edge).
- **Rewards**:
  - **-1** for each step to encourage the shortest path.
  - **+10** for reaching the goal `(3, 3)`.
- **Objective**: Train the agent to find the optimal policy using SARSA.

In [8]:

```
import numpy as np  # Import NumPy for numerical operations
import random  # Import random module for generating random numbers

# Step 1: Define the GridWorld Environment
class GridWorld:
    def __init__(self, size=4):
        """
        Initialize the GridWorld environment.

        Parameters:
        - size: The size of the grid (default is 4x4).
        """
        self.size = size  # Set the grid size
        self.start = (0, 0)  # Starting position of the agent
        self.goal = (3, 3)  # Goal position for the agent
        self.state = self.start  # Initialize the agent's current s

    def reset(self):
        """
        Reset the environment to its initial state.

        Returns:
        - The starting position of the agent.
        """
        self.state = self.start  # Reset the agent's state to the s
        return self.state  # Return the starting state

    def step(self, action):
        """
        Execute an action and return the next state, reward, and wh

        Parameters:
        - action: The action to be taken ('up', 'down', 'left', 'ri

        Returns:
        - next_state: The new state after the action.
```

```
        - reward: The reward for taking the action.
        - done: Whether the episode has ended (True if the goal is
        """
        row, col = self.state  # Extract the current position (row,

        # Move the agent based on the action
        if action == "up" and row > 0:  # Move up if not at the top
            row -= 1
        elif action == "down" and row < self.size - 1:  # Move down
            row += 1
        elif action == "left" and col > 0:  # Move left if not at t
            col -= 1
        elif action == "right" and col < self.size - 1:  # Move rig
            col += 1

        # Update the agent's position
        next_state = (row, col)

        # Check if the agent has reached the goal
        if next_state == self.goal:
            reward = 10  # Reward for reaching the goal
            done = True  # End the episode
        else:
            reward = -1  # Penalty for each step
            done = False  # Continue the episode

        self.state = next_state  # Update the current state to the
        return next_state, reward, done  # Return the new state, re

# Step 2: Implement the SARSA Algorithm
def sarsa(env, episodes, alpha=0.1, gamma=0.9, epsilon=0.1):
    """
    Train an agent using the SARSA algorithm.

    Parameters:
    - env: The GridWorld environment.
    - episodes: The number of training episodes.
    - alpha: The learning rate.
    - gamma: The discount factor.
    - epsilon: The exploration rate.

    Returns:
    - Q: The learned Q-table (state-action values).
    """
    actions = ["up", "down", "left", "right"]  # Define possible ac
    Q = {}  # Initialize the Q-table

    # Initialize Q-values for all state-action pairs
    for row in range(env.size):
        for col in range(env.size):
            Q[(row, col)] = {action: 0 for action in actions}  # Se

    # Training loop for the specified number of episodes
    for episode in range(episodes):
        state = env.reset()  # Reset the environment at the start o
        action = random.choice(actions)  # Choose an initial action
```

```
        done = False  # Initialize the "done" flag as False

        while not done:
            # Take the action and observe the next state, reward, a
            next_state, reward, done = env.step(action)

            # Choose the next action using epsilon-greedy policy
            if random.uniform(0, 1) < epsilon:  # Exploration: Choo
                next_action = random.choice(actions)
            else:  # Exploitation: Choose the action with the highe
                next_action = max(Q[next_state], key=Q[next_state].

            # Update the Q-value for the current state-action pair
            Q[state][action] += alpha * (reward + gamma * Q[next_st

            # Transition to the next state and action
            state, action = next_state, next_action

    return Q  # Return the learned Q-table

# Step 3: Train the SARSA Agent
env = GridWorld(size=4)  # Create a 4x4 GridWorld environment
episodes = 500  # Define the number of episodes for training
Q = sarsa(env, episodes, alpha=0.1, gamma=0.9, epsilon=0.1)  # Trai

# Step 4: Display the Learned Q-Values
print("Learned Q-Values:")
for state, actions in Q.items():
    print(f"State {state}: {actions}")
```

```
Learned Q-Values:
State (0, 0): {'up': -0.5214923768498055, 'down': 0.743707459256133
4, 'left': -0.6788505655146715, 'right': 0.186685123468751}
State (0, 1): {'up': -1.5990475736266987, 'down': 1.755296664529905,
'left': -1.2653297835163513, 'right': -1.3621043724582387}
State (0, 2): {'up': -0.939087077587669, 'down': -0.821688204255422,
'left': -0.8997520837821265, 'right': 0.7594909305829204}
State (0, 3): {'up': -0.56376852367591, 'down': 3.9605030040841562,
'left': -0.5401603502479, 'right': -0.21560799859085583}
State (1, 0): {'up': -1.0095975432312603, 'down': 2.205457280674986
4, 'left': -0.585909189972644, 'right': -1.0177918304179343}
State (1, 1): {'up': -0.9767538077833446, 'down': -0.025619450733065
93, 'left': -0.38242544737345274, 'right': 2.7195907566908497}
State (1, 2): {'up': -0.965686088066044, 'down': 0.7332003352215684,
'left': -0.5451106308501774, 'right': 4.049030645464304}
State (1, 3): {'up': 0.3651664928231807, 'down': 7.7512103922123075,
'left': 0.34520137414083835, 'right': 1.2526119917492764}
State (2, 0): {'up': -0.09024540337936555, 'down': 0.263658822132339
8, 'left': 0.40624897207585486, 'right': 3.775763578869169}
State (2, 1): {'up': 0.8875727312942687, 'down': 5.213297021510254,
'left': -0.1559946416212039, 'right': 0.3094191498706484}
State (2, 2): {'up': 0.3542949621750838, 'down': 0.6289999999906148,
'left': -0.10900000000000001, 'right': 6.629878252412003}
State (2, 3): {'up': 1.144140994439652, 'down': 9.999999771703592, '
left': 0.7123392851923122, 'right': 3.6357889194944337}
State (3, 0): {'up': -0.52600660741, 'down': -0.55931149, 'left': -
0.5629712814100001, 'right': 4.067795541986756}
State (3, 1): {'up': 1.8043618764839557, 'down': 3.21490058837726, '
left': 0.18655768743314505, 'right': 7.395123209019041}
State (3, 2): {'up': 1.1486478747453353, 'down': 4.105237467024043,
'left': 2.3046385739826167, 'right': 9.999999999999993}
State (3, 3): {'up': 0, 'down': 0, 'left': 0, 'right': 0}
```

Interpreting the Q-Values Output

The **Q-values** represent the learned state-action values for a reinforcement learning agent navigating an environment. Each state (e.g., `(0, 0)`) has a set of Q-values for all possible actions (`'up'`, `'down'`, `'left'`, `'right'`). These values indicate the agent's expected cumulative reward for taking a specific action in a given state and following the optimal policy thereafter.

General Observations

1. **Positive Q-Values**:
    - Actions with positive Q-values represent favorable choices, leading the agent closer to the goal or yielding rewards.
2. **Negative Q-Values**:
    - Actions with negative Q-values indicate penalties, inefficiencies, or

actions leading to less optimal outcomes.

3. **Zero Q-Values (e.g., State (3, 3))**:
   - At terminal states, such as `(3, 3)`, all Q-values are zero because no further actions are needed (the episode ends).

Detailed State Analysis

State (0, 0):

```
{'up': -0.521, 'down': 0.744, 'left': -0.679, 'right':
0.187}
```

- **Interpretation**:
  - **'down'** has the highest Q-value (`0.744`), making it the best action from this state.
  - **'up'** and **'left'** have negative Q-values, indicating they lead to penalties.
  - **'right'** has a small positive Q-value, making it a slightly favorable alternative.
- **What it means**:
  - The agent prefers moving **down** from `(0, 0)` to maximize rewards and minimize penalties.

State (0, 3):

```
{'up': -0.564, 'down': 3.961, 'left': -0.540, 'right':
-0.216}
```

- **Interpretation**:
  - **'down'** has the highest Q-value (`3.961`), indicating it leads closer to the goal.
  - All other actions have negative Q-values, representing suboptimal moves.
- **What it means**:
  - From `(0, 3)`, the agent chooses **'down'** to progress effectively toward the goal.

State (1, 3):

```
{'up': 0.365, 'down': 7.751, 'left': 0.345, 'right': 1.253}
```

- **Interpretation**:
  - **'down'** has the highest Q-value (`7.751`), suggesting it leads directly to the goal.
  - Other actions have lower but positive Q-values, showing they are less

effective but not harmful.

- **What it means**:
  - The agent strongly prefers moving **down** from `(1, 3)` to reach the goal efficiently.

State (2, 3):

```
{'up': 1.144, 'down': 10.000, 'left': 0.712, 'right': 3.636}
```

- **Interpretation**:
  - **'down'** has the highest Q-value (`10.000`), leading directly to the goal.
  - **'up'** and **'right'** are alternative options with positive Q-values, but less rewarding.
  - **'left'** is the least favorable action, with the lowest Q-value.
- **What it means**:
  - The agent confidently chooses **'down'** to reach the goal.

State (3, 2):

```
{'up': 1.149, 'down': 4.105, 'left': 2.305, 'right': 10.000}
```

- **Interpretation**:
  - **'right'** has the highest Q-value (`10.000`), indicating it directly leads to the goal.
  - Other actions have lower but positive Q-values, representing less optimal alternatives.
- **What it means**:
  - From `(3, 2)`, the agent moves **right** to complete the episode.

Key Insights

1. **Optimal Actions**:
   - The action with the highest Q-value in each state represents the **optimal policy**. The agent follows this policy to maximize cumulative rewards.
2. **Goal-Oriented Behavior**:
   - States closer to the goal (e.g., `(2, 3)` and `(3, 2)`) have higher Q-values for the optimal actions, reflecting the increasing rewards as the agent nears the goal.
3. **Penalties**:
   - Negative Q-values in some states and actions show that the agent

learns to avoid these actions due to penalties incurred along those paths.

4. **Exploration vs. Exploitation**:
    - Positive Q-values for suboptimal actions (e.g., **'up'** or **'left'** in `(1, 3)`) reflect that the agent occasionally explored these moves but ultimately found better alternatives.

What Do We Achieve?

- **Dynamic Learning**:
    - The agent has learned an effective navigation strategy through trial and error using the SARSA algorithm.
- **Optimal Policy**:
    - For each state, the optimal action can be determined by selecting the action with the highest Q-value.
- **Risk-Aware Behavior**:
    - The agent avoids unnecessary penalties and prioritizes actions leading to higher rewards.

Practical Significance

1. **Navigation Problems**:
    - This approach can train robots or autonomous systems to navigate safely and efficiently in grid-like environments.
2. **Gaming AI**:
    - SARSA is used to create intelligent game agents that learn and adapt to different environments.
3. **Logistics**:
    - Similar techniques can optimize warehouse robot paths or delivery routes to minimize time and costs.

Industry-Oriented SARSA Example: Optimizing Energy Consumption in a Smart Thermostat

Problem Statement

In a smart home, a thermostat adjusts the heating or cooling system to maintain a comfortable temperature while minimizing energy consumption. The thermostat has multiple states (representing current temperature levels) and actions (adjusting heating, cooling, or maintaining the temperature).

The goal is to train the thermostat using **SARSA** to find the optimal policy that balances comfort and energy efficiency.

Environment Setup

- **States**: Represent different temperature levels (e.g., 18°C, 20°C, 22°C, 24°C).
- **Actions**:
  - **Heat**: Increase temperature by 2°C.
  - **Cool**: Decrease temperature by 2°C.
  - **Maintain**: Keep the current temperature.
- **Rewards**:
  - Positive reward for maintaining the desired range (e.g., 20°C to 22°C).
  - Negative reward for temperatures outside the desired range (e.g., energy inefficiency or user discomfort).
- **Objective**: Train the thermostat to maintain a comfortable temperature range while minimizing energy use.

In [1]:
```
import numpy as np  # For numerical operations
import random  # For random number generation

# Step 1: Define the Smart Thermostat Environment
class SmartThermostat:
    def __init__(self, min_temp=18, max_temp=24, desired_range=(20,
        """
        Initialize the thermostat environment.

        Parameters:
        - min_temp: Minimum temperature the system can reach.
        - max_temp: Maximum temperature the system can reach.
        - desired_range: Temperature range considered comfortable (
        - max_steps: Maximum number of steps per episode.
        """
        self.min_temp = min_temp  # Minimum temperature
        self.max_temp = max_temp  # Maximum temperature
        self.desired_range = desired_range  # Comfortable temperatu
        self.state = random.randint(min_temp, max_temp)  # Start at
        self.max_steps = max_steps  # Limit the number of steps per

    def reset(self):
        """
        Reset the environment to a random initial temperature.

        Returns:
```

```
        - The initial temperature state.
        """
        self.state = random.randint(self.min_temp, self.max_temp)
        self.steps = 0  # Reset step counter
        return self.state

    def step(self, action):
        """
        Execute an action and return the next state, reward, and wh

        Parameters:
        - action: Action to perform ('heat', 'cool', 'maintain').

        Returns:
        - next_state: The resulting temperature.
        - reward: Reward for the action.
        - done: Whether the episode is over.
        """
        # Increment step counter
        self.steps += 1

        # Apply the action to change the temperature
        if action == "heat":
            self.state = min(self.state + 2, self.max_temp)  # Incr
        elif action == "cool":
            self.state = max(self.state - 2, self.min_temp)  # Decr
        elif action == "maintain":
            pass  # No change in temperature

        # Calculate the reward
        if self.desired_range[0] <= self.state <= self.desired_rang
            reward = 10  # Positive reward for staying in the desir
        else:
            reward = -1  # Penalty for being outside the desired ra

        # Determine if the episode should terminate
        done = self.steps >= self.max_steps  # Terminate if maximum

        return self.state, reward, done  # Return next state, rewar

# Step 2: Implement the SARSA Algorithm
def sarsa(env, episodes, alpha=0.1, gamma=0.9, epsilon=0.1):
    """
    Train the smart thermostat using SARSA.

    Parameters:
    - env: The SmartThermostat environment.
    - episodes: Number of training episodes.
    - alpha: Learning rate.
    - gamma: Discount factor.
    - epsilon: Exploration rate.

    Returns:
    - Q: The learned Q-table.
    """
    actions = ["heat", "cool", "maintain"]  # Define possible actio
```

```
    Q = {}  # Initialize Q-table

    # Initialize Q-values for all state-action pairs
    for temp in range(env.min_temp, env.max_temp + 1):
        Q[temp] = {action: 0 for action in actions}  # Set Q-values

    # Training loop
    for episode in range(episodes):
        state = env.reset()  # Reset environment at the start of ea
        action = random.choice(actions)  # Choose an initial action
        done = False  # Initialize the "done" flag as False

        while not done:
            next_state, reward, done = env.step(action)  # Take the

            # Choose the next action using epsilon-greedy policy
            if random.uniform(0, 1) < epsilon:  # Exploration: Choo
                next_action = random.choice(actions)
            else:  # Exploitation: Choose the action with the highe
                next_action = max(Q[next_state], key=Q[next_state].

            # Update the Q-value for the current state-action pair
            Q[state][action] += alpha * (reward + gamma * Q[next_st

            # Transition to the next state and action
            state, action = next_state, next_action

    return Q  # Return the learned Q-table

# Step 3: Train the SARSA Agent
env = SmartThermostat()  # Create the smart thermostat environment
episodes = 500  # Number of episodes for training
Q = sarsa(env, episodes, alpha=0.1, gamma=0.9, epsilon=0.1)  # Trai

# Step 4: Display Results
print("Learned Q-Values:")
for state, actions in Q.items():
    print(f"Temperature {state}: {actions}")
```

```
Learned Q-Values:
Temperature 18: {'heat': 96.20346877132279, 'cool': 84.8731887874195
8, 'maintain': 83.9359732985471}
Temperature 19: {'heat': 91.12389975297681, 'cool': 83.5281071998794
9, 'maintain': 79.35783465356235}
Temperature 20: {'heat': 95.69712745522753, 'cool': 85.4060488985648
9, 'maintain': 96.05021339956555}
Temperature 21: {'heat': 81.313150924503, 'cool': 81.5438103384224,
'maintain': 92.24294589138451}
Temperature 22: {'heat': 84.1636298443716, 'cool': 96.1439890456646
7, 'maintain': 94.75724908724654}
Temperature 23: {'heat': 80.51967063158581, 'cool': 92.151705193178
8, 'maintain': 76.14568238125864}
Temperature 24: {'heat': 85.11271828406124, 'cool': 96.4807892172363
9, 'maintain': 83.20461474883545}
```

Interpreting the Learned Q-Values

The output represents the **Q-values** for a smart thermostat system trained using **SARSA** to optimize energy consumption while maintaining comfort. Each temperature state (18°C to 24°C) has three possible actions:

- `heat`: Increase the temperature.
- `cool`: Decrease the temperature.
- `maintain`: Keep the temperature unchanged.

The Q-value for each action indicates the expected cumulative reward if that action is taken in the given state, considering the learned policy.

Detailed Interpretation

1. **Temperature 18°C**:
   - `heat`**: 96.20** is the highest Q-value.
   - This suggests that increasing the temperature (heating) from 18°C is the optimal action to reach the comfortable range (20°C–22°C).
   - `cool` and `maintain` have lower Q-values, meaning they are suboptimal actions in this state.
2. **Temperature 19°C**:
   - `heat`**: 91.12** is the highest Q-value.
   - The agent learns that heating is still the best choice to bring the temperature closer to the desired range.
   - The Q-values for `cool` and `maintain` are lower, as these actions would not lead to rewards.
3. **Temperature 20°C**:
   - `maintain`**: 96.05** is the highest Q-value.
   - This indicates that staying at 20°C is optimal because it is within the desired range (20°C–22°C).
   - `heat` and `cool` have lower Q-values, reflecting unnecessary energy expenditure and a potential penalty for moving out of the range.
4. **Temperature 21°C**:
   - `maintain`**: 92.24** is the highest Q-value.
   - Similar to 20°C, maintaining the current temperature is the best action as 21°C is within the desired range.
   - `heat` and `cool` have lower Q-values, meaning they are less effective.
5. **Temperature 22°C**:
   - `cool`**: 96.14** is the highest Q-value.

- Cooling is preferred here, as it brings the temperature closer to the middle of the desired range.
- `maintain` also has a high Q-value, indicating that keeping the temperature unchanged is still a good option.
- `heat` is less optimal, as it moves the temperature further from the desired range.

6. **Temperature 23°C**:
   - `cool`**: 92.15** is the highest Q-value.
   - Cooling is the optimal action to bring the temperature back into the desired range.
   - `maintain` has a much lower Q-value, as staying at 23°C results in a penalty for being outside the range.
7. **Temperature 24°C**:
   - `cool`**: 96.48** is the highest Q-value.
   - Similar to 23°C, cooling is the best choice to reduce the temperature to the desired range.
   - `heat` and `maintain` are less effective, with lower Q-values.

Why This Output Makes Sense

1. **Logical Action Selection**:
   - For temperatures below the desired range (18°C–19°C), `heat` has the highest Q-value.
   - For temperatures within the desired range (20°C–22°C), `maintain` has the highest Q-value.
   - For temperatures above the desired range (23°C–24°C), `cool` has the highest Q-value.
2. **Cumulative Reward**:
   - Q-values reflect the long-term benefit of an action, considering future rewards and penalties. The highest Q-value in each state corresponds to the optimal action for maximizing rewards.
3. **Dynamic Learning**:
   - The agent adapts to different temperature states and learns to balance energy efficiency and comfort.

What We Achieve

1. **Optimal Policy**:

- The learned Q-values provide a clear policy for the thermostat:
  - Heat when too cold.
  - Maintain when in the comfortable range.
  - Cool when too hot.

2. **Energy Efficiency**:
   - The thermostat avoids unnecessary actions that waste energy while maintaining user comfort.
3. **Practical Implementation**:
   - This approach can be extended to real-world scenarios such as HVAC systems, industrial cooling, or home automation.

Refer Exercise NB - Stub file

Happy Learning